

Inhaltsverzeichnis

Inhaltsverzeichnis	I
1 Entwicklungsumgebung für Kurs	2
1.1 Anaconda Navigator	4
1.2 Conda-Umgebung einrichten	5
1.3 Spyder Editor	9
Literaturverzeichnis	11

1 Entwicklungsumgebung für Kurs

Wie eine Entwicklungsumgebung gestaltet wird, hängt wesentlich davon ab, welche Werkzeuge Verwendung finden sollen. Im Folgenden wird vorgestellt, wie die Installation erfolgt, wenn Python (Python Software Foundation, 2026) in Verbindung mit der Integrated Development Environment (IDE) von **Spyder** (Spyder, 2026) genutzt werden soll. Besonders geschickt ist die Installation der obigen Werkzeuge mit Hilfe des Distributionspakets **Anaconda**, welche die Programmiersprachen Python und die Entwicklungsumgebung Spyder enthält. Das Anaconda Distributionspaket kann unter folgender URL <https://www.anaconda.com/download> (Anaconda, 2026) heruntergeladen werden (siehe Abb. 1-1).

Abb. 1-1: Get Started -> Download

Entscheiden Sie sich für die Datei „Anaconda3-2025.12-2-Windows-x86_64 (Windows, 64-Bit)“. Wenn Sie auf „Download“ klicken, können Sie die Installationsdatei speichern oder auch direkt ausführen. Bestätigen Sie, dass Sie die Installation ausführen möchten (siehe Abb. 1-2).

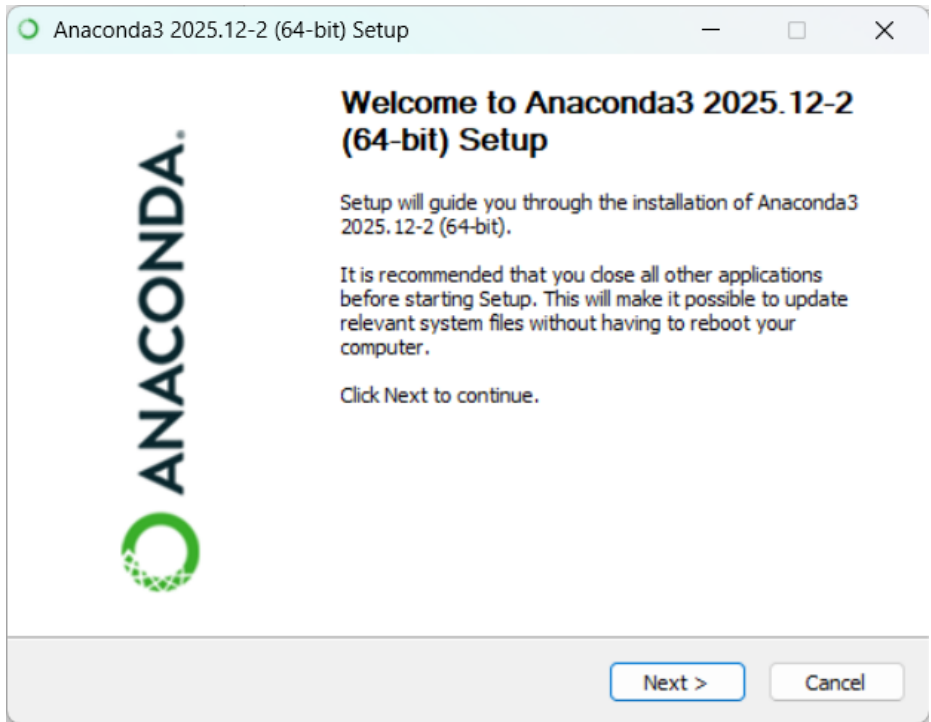


Abb. 1-2: Setup

Klicken Sie auf „Next“, um das Setup auszuführen. Stimmen Sie den Lizenzbedingungen zu, wählen das Installationsverzeichnis aus und registrieren Sie Anaconda3 als Ihre Standard-Python Umgebung. Achten Sie darauf, dass Sie Anaconda3 NICHT zu Ihrem PATH hinzufügen (nicht empfohlen).

Nach der Installation wird diese als „completed“ bestätigt. Eine weitere Bestätigung wird angezeigt, nachdem Sie auf „Next“ geklickt haben. Auf dem nächsten Fenster können Sie die Webseite mit „Getting Started“ öffnen. Dies können Sie bestätigen und fortfahren. Auf der angezeigten Webseite finden Sie Tutorials und die Dokumentation zu Anaconda. Sie können jetzt alle Browser-Fenster schließen. Wenn Sie auf in Windows nach Anaconda

suchen, dann werden Ihnen die neu installierten Applikationen angezeigt: Anaconda Navigator und Anaconda PowerShell Prompt (siehe Abb. 1-3).

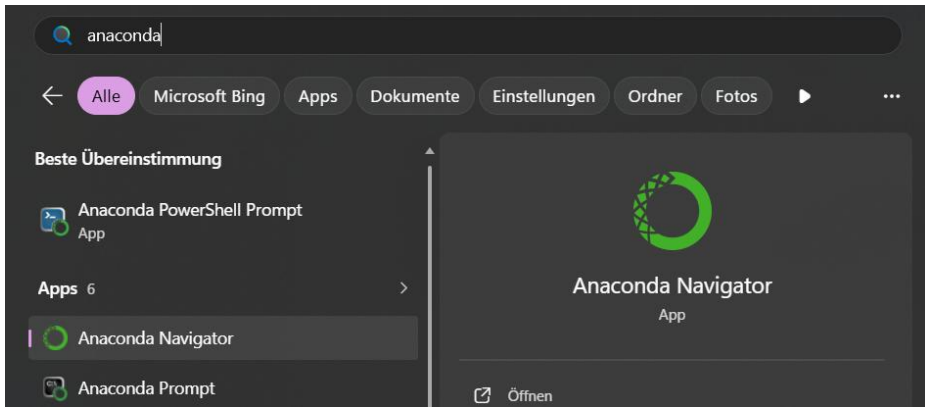


Abb. 1-3: Anaconda Apps

1.1 Anaconda Navigator

Starten Sie auch den Anaconda Navigator (siehe Abb. 1-4). Sie könnten hier noch weitere Programme installieren. Dies ist jedoch nicht erforderlich. Verlassen Sie den Navigator daher wieder.

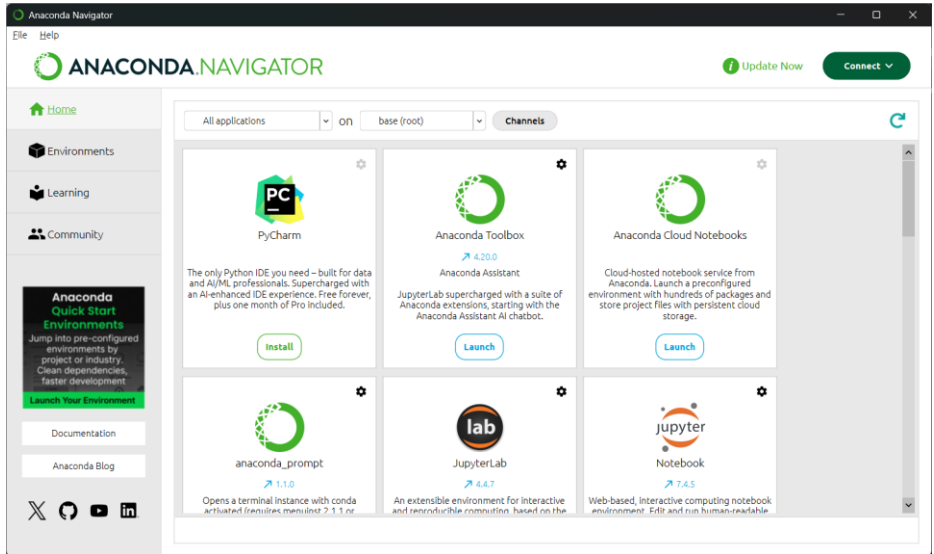


Abb. 1-4: Anaconda Navigator

Führen Sie jetzt einen Neustart Ihres Rechners aus, um u.a. die PATH-Einstellungen wirksam zu machen.

1.2 Conda-Umgebung einrichten

Starten Sie die App Anaconda PowerShell Prompt und lassen Sie sich die Version von Conda und von Python anzeigen mit den Anweisungen: `conda -- version` und `python -- version` (siehe Abb. 1-5).

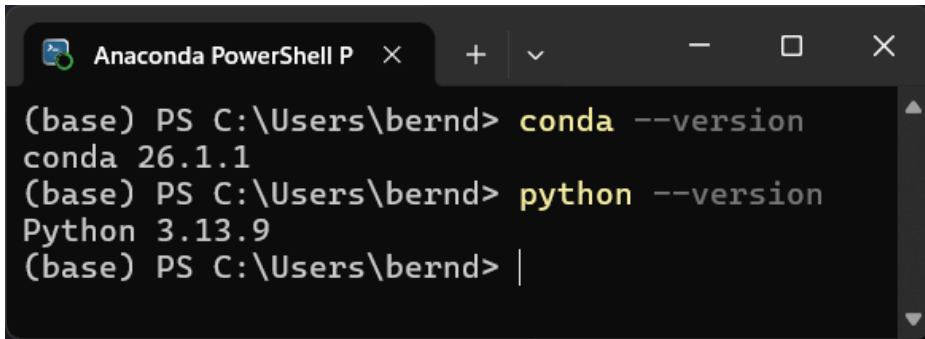
A screenshot of an Anaconda PowerShell terminal window. The window title is "Anaconda PowerShell P". The terminal shows three commands and their outputs: 1. Command: `conda --version`, Output: `conda 26.1.1`. 2. Command: `python --version`, Output: `Python 3.13.9`. 3. Command: `(base) PS C:\Users\bernd>` followed by a cursor. The terminal has a dark background with light-colored text.

Abb. 1-5: Anaconda PowerShell

Um für die nachfolgende Codebeispiele eine einheitliche Umgebung zu erstellen (Pakete mit Versionen, welche miteinander kompatibel sind) sollte eine Conda-Umgebung „torch-cpu“ eingerichtet werden, die Python, scikit-learn, PyTorch (PyTorch, 2026) und einige weitere Pakete beinhaltet.

Erstellen Sie eine Datei mit dem Namen „torch-cpu_environment.yml“ mit folgendem Inhalt und speichern diese, z.B. in dem Verzeichnis „Downloads“. Sie können sich die Datei auch von der GitHub-Seite zu dem Buch unter <https://github.com/bheesen/pythonforbusiness2> im Verzeichnis Installation herunterladen (Heesen, 2026).

Torch-CPU

```
name: torch-cpu

# Zielbasis: Anaconda Distribution 2025.12-2 unter Windows 11
# Reproduzierbare CPU-Umgebung mit zueinander passenden PyTorch-Paketen.
# PyTorch 2.11.0 ist die Letzte hier verwendete gemeinsame Version von
# torch, torchvision und torchaudio.

channels:
  - conda-forge
  - nodefaults

dependencies:
  # =====
  # 1) Python und Paketverwaltung
  # =====
  - python=3.13
  - pip
  - setuptools
  - wheel
```

```
# =====
# 2) Wissenschaftlicher Kern
# =====
- numpy=2.5
- scipy
- pandas
- statsmodels
- scikit-learn=1.9
- joblib

# =====
# 3) Visualisierung
# =====
- matplotlib
- seaborn
- plotly
- folium

# =====
# 4) Datenimport, Dateiformate und grosse Datenmengen
# =====
- openpyxl
- pyarrow
- pyreadr
- dask
- distributed

# =====
# 5) Jupyter und interaktives Arbeiten
# =====
- jupyterlab
- notebook
- ipykernel
- ipython

# =====
# 6) Spyder-Anbindung
#   Spyder wird aus base gestartet.
# =====
- "spyder-kernels>=3.1,<3.2"

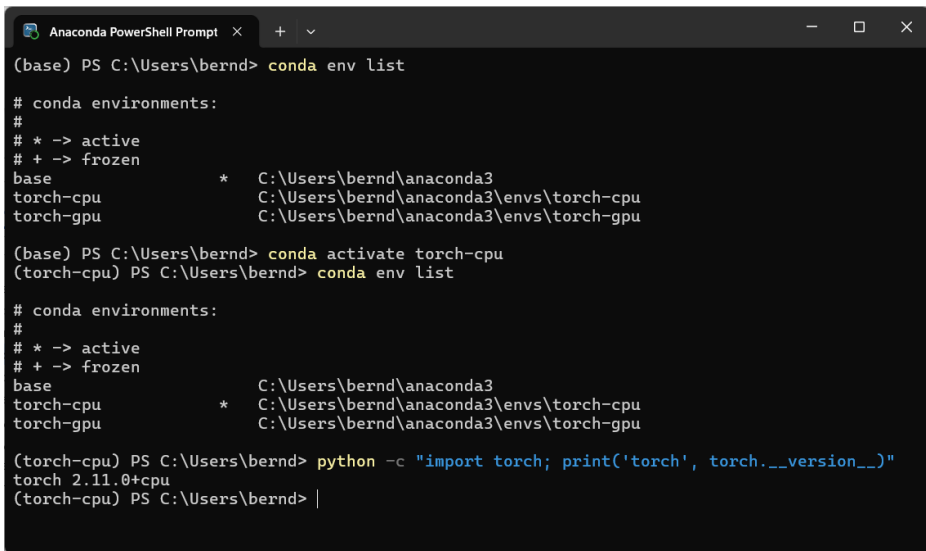
# =====
# 7) Hilfsprogramme und Systeminformationen
# =====
- py-cpuinfo
- psutil
- tqdm
- filelock
- typing_extensions
- sympy
- networkx
- jinja2
- fsspec
- pillow

# =====
# 8) PyTorch-Stack - ausschliesslich CPU
#   Die drei Versionen sind aufeinander abgestimmt.
# =====
```

```
- pip:
- --index-url https://download.pytorch.org/whl/cpu
- torch==2.11.0
- torchvision==0.26.0
- torchaudio==2.11.0
```

Öffnen Sie den Anaconda PowerShell Prompt und wechseln Sie in den Ordner mit der yml-Datei, z. B.: `cd %USERPROFILE%\Downloads`. Erstellen Sie dann die Conda-Umgebung mit der Anweisung `conda env create -f torch-cpu_environment.yml`.

Sie können sich alle Conda-Umgebungen mit der Anweisung `conda env list` anzeigen lassen, wobei die aktive Umgebung mit einem * gekennzeichnet ist. Um z.B. die neu erstellte Umgebung `torch-cpu` auch als aktive Umgebung auszuwählen können Sie die Anweisung `conda activate torch-cpu` verwenden. Um zu testen, ob dort das Paket PyTorch installiert wurde, können Sie die folgende Anweisung ausführen: `python -c "import torch; print('torch', torch.__version__)"`. Es sollte Ihnen die installierte Version des Pakets anzeigen (siehe Abb. 1-6).



```
Anaconda PowerShell Prompt
(base) PS C:\Users\bernd> conda env list

# conda environments:
#
# * -> active
# + -> frozen
base                * C:\Users\bernd\anaconda3
torch-cpu           C:\Users\bernd\anaconda3\envs\torch-cpu
torch-gpu           C:\Users\bernd\anaconda3\envs\torch-gpu

(base) PS C:\Users\bernd> conda activate torch-cpu
(torch-cpu) PS C:\Users\bernd> conda env list

# conda environments:
#
# * -> active
# + -> frozen
base                C:\Users\bernd\anaconda3
torch-cpu           * C:\Users\bernd\anaconda3\envs\torch-cpu
torch-gpu           C:\Users\bernd\anaconda3\envs\torch-gpu

(torch-cpu) PS C:\Users\bernd> python -c "import torch; print('torch', torch.__version__)"
torch 2.11.0+cpu
(torch-cpu) PS C:\Users\bernd> |
```

Abb. 1-6: Details der Umgebung `torch-cpu`

Um diese Umgebung später auch in der IDE Spyder verwenden zu können, gilt es diese Umgebung noch als die Standardumgebung festzulegen.

1.3 Spyder Editor

Klicken Sie auf Spyder, um den Python Editor zu öffnen. Wählen Sie in Spyder unter Werkzeuge/Einstellungen den Python Interpreter, der in dem Verzeichnis torch-cpu installiert ist. Klicken Sie dazu auf das Dropdown-Symbol und bestätigen die gewünschte Umgebung (siehe Abb. 1-7).

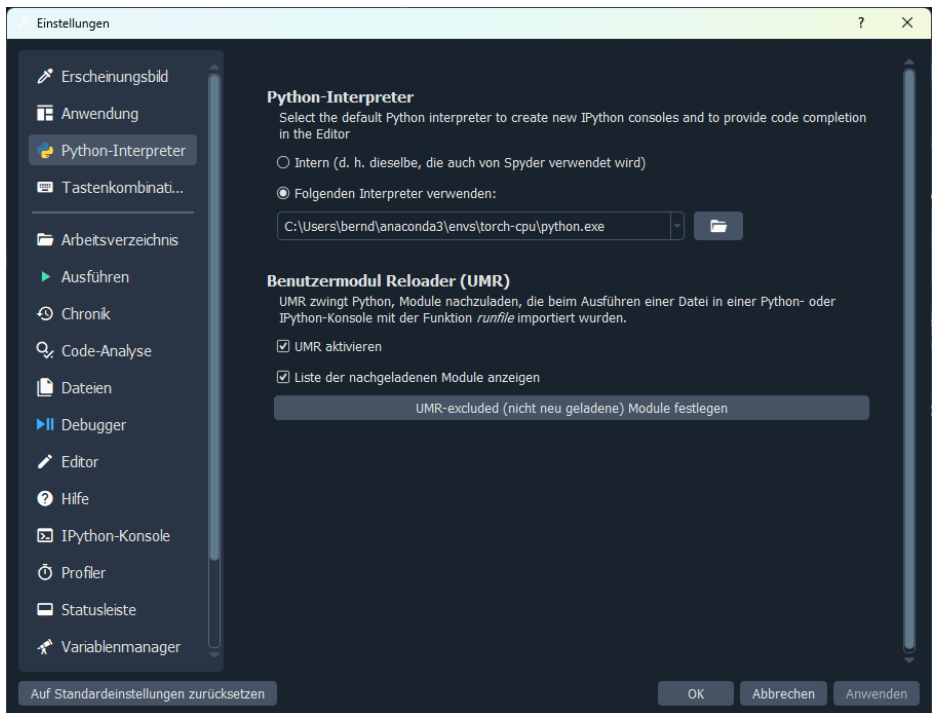
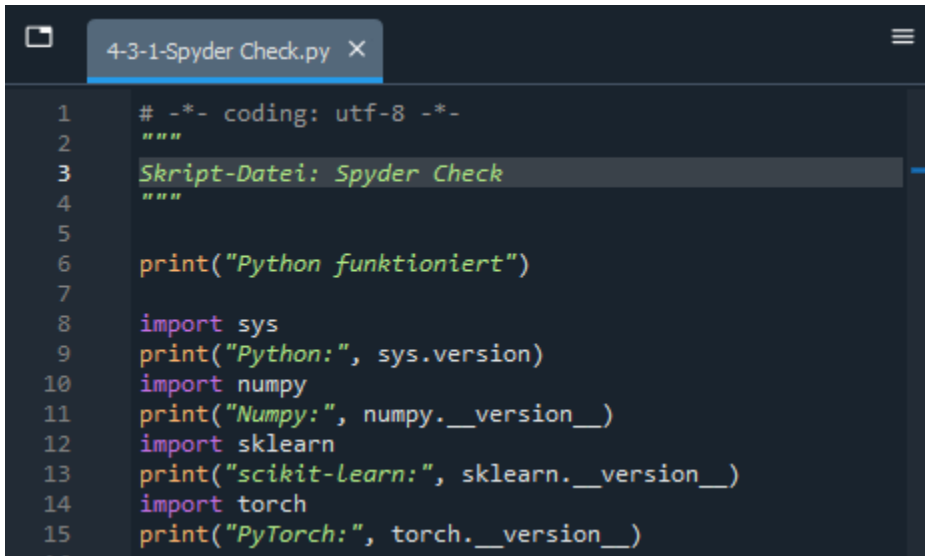


Abb. 1-7: Spyder Preferences: Python Interpreter

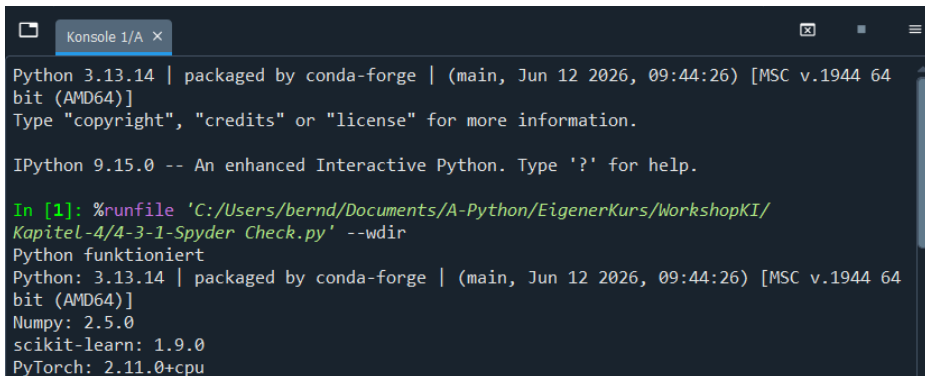
Schließen Sie Spyder und starten Spyder anschließend erneut, damit die Änderungen wirksam werden. Legen Sie jetzt die Skriptdatei „4-3-1-Spyder Check.py“ an (siehe Abb. 1-8).

A screenshot of a Spyder Python IDE window. The title bar shows '4-3-1-Spyder Check.py'. The code editor contains a Python script with the following content:

```
1  # -*- coding: utf-8 -*-
2  """
3  Skript-Datei: Spyder Check
4  """
5
6  print("Python funktioniert")
7
8  import sys
9  print("Python:", sys.version)
10 import numpy
11 print("Numpy:", numpy.__version__)
12 import sklearn
13 print("scikit-learn:", sklearn.__version__)
14 import torch
15 print("PyTorch:", torch.__version__)
```

Abb. 1-8: Python Skript

Wenn Sie den Code ausführen, indem Sie auf F5 oder „Run file“ klicken, dann sollte die Ausgabe in der Konsole erscheinen (siehe Abb. 1-9).

A screenshot of a Python console window titled 'Konsole 1/A'. It shows the output of running the script from the previous image. The output includes the Python version, package information, and the results of the print statements in the script.

```
Python 3.13.14 | packaged by conda-forge | (main, Jun 12 2026, 09:44:26) [MSC v.1944 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 9.15.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: %runfile 'C:/Users/bernd/Documents/A-Python/EigenerKurs/WorkshopKI/
Kapitel-4/4-3-1-Spyder Check.py' --wdir
Python funktioniert
Python: 3.13.14 | packaged by conda-forge | (main, Jun 12 2026, 09:44:26) [MSC v.1944 64
bit (AMD64)]
Numpy: 2.5.0
scikit-learn: 1.9.0
PyTorch: 2.11.0+cpu
```

Abb. 1-9: Python Editor

Damit haben Sie Ihr erstes Python-Programm erstellt und erfolgreich getestet.

Literaturverzeichnis

Anaconda. (2026). Abgerufen am 28. Juni 2026 von <https://www.anaconda.com/download>

Heesen, B. (2026). *GitHub: bheesen/pythonforbusiness2*. Abgerufen am 29. Juni 2026 von <https://github.com/bheesen/pythonforbusiness2>

Python Software Foundation. (2026). *Python*. Abgerufen am 20. Januar 2026 von <https://www.python.org>

PyTorch. (2026). Abgerufen am 28. Juni 2026 von <https://pytorch.org/>

Spyder. (2026). *Spyder: The scientific Python development environment*. Abgerufen am 30. Juni 2026 von <https://www.spyder-ide.org/>